Sesión 1

Copy

Curso de Azure AI Foundry, MAF y Azure OpenAI para Arquitecturas Avanzadas de Agentes IA

Contenidos de Hoy

1. Presentación del Curso
2. Entorno Operativo
3. Agentes en Azure
4. Foundry
5. Descanso
6. Azure OpenAI
7. Preparación del Entorno
8. Recap y Dudas
9. Cuestionario

Presentación del Curso

Calendario y Temario

1MB
Imagina Azure AI Temario Mar-26.pdf
PDF
Download Open

Objetivos

- Diseñar Agentes usando **Azure OpenAI, Microsoft Foundry y Microsoft Agent Framework**.
- Migrar arquitecturas basadas en **LangChain y LangGraph**
- Desarrollar **Arquitecturas Agénticas** avanzadas: multiagente, tools, workflows, memoria, RAG, observabilidad, evaluación y guardrails.
- **Integrar y Operar** agentes avanzados, incluyendo **Copilot Studio, Agent 265 y Frontier**.
- Entregar una **solución final** basada un un caso de uso real.

Metodología

- Trabajo en Azure, Desarrollo en **Python** bajo VSCode
- **Desarrollo teórico escaso**, pero importante tener claros algunos conceptos importantes.
- Yo comienzo los ejercicios, **vosotros los terminái**s.
- **Participación activa** mostrando pantalla, vía chat o interrumpiendo.
- Ejercicios **individuales**. Mejor no usar IA :-)
- **Cuestionario** de fin de sesión y cuestionario de feedback.
- **Proyecto Final**

INTRODUCCIÓN A ARQUITECTURAS AVANZADAS DE
AGENTES EN AZURE

Evolución hacia sistemas agénticos enterprise más allá del chatbot simple o del agente aislado

Un **chatbot simple responde bien preguntas directas**, pero normalmente:

- no mantiene un estado útil a lo largo del tiempo,
- no coordina varias capacidades,
- no opera con gobierno,
- no tiene trazabilidad suficiente,
- y no está preparado para procesos empresariales reales.

El problema no aparece en la demo. Aparece cuando el sistema tiene que:

- consultar varios orígenes de datos,
- ejecutar acciones sobre sistemas corporativos,
- aplicar permisos,
- justificar decisiones,
- soportar fallos parciales,
- y operar a escala.

Qué cambia en enterprise

En entorno enterprise ya no hablamos de “un bot que conversa”, sino de un **sistema socio-técnico** con varias capas:

- capa de experiencia,
- capa de orquestación,
- capa de modelos,
- capa de tools,
- capa de datos,
- capa de seguridad,
- capa de observabilidad y evaluación.

El valor no está solo en generar texto. Está en:

- **tomar decisiones acotadas**,
- **invocar herramientas seguras**,
- **trabajar con contexto corporativo**,
- **respetar políticas**,
- **dejar evidencias operativas**.

Por qué el agente aislado se queda corto

Un agente aislado suele mezclar demasiadas responsabilidades:

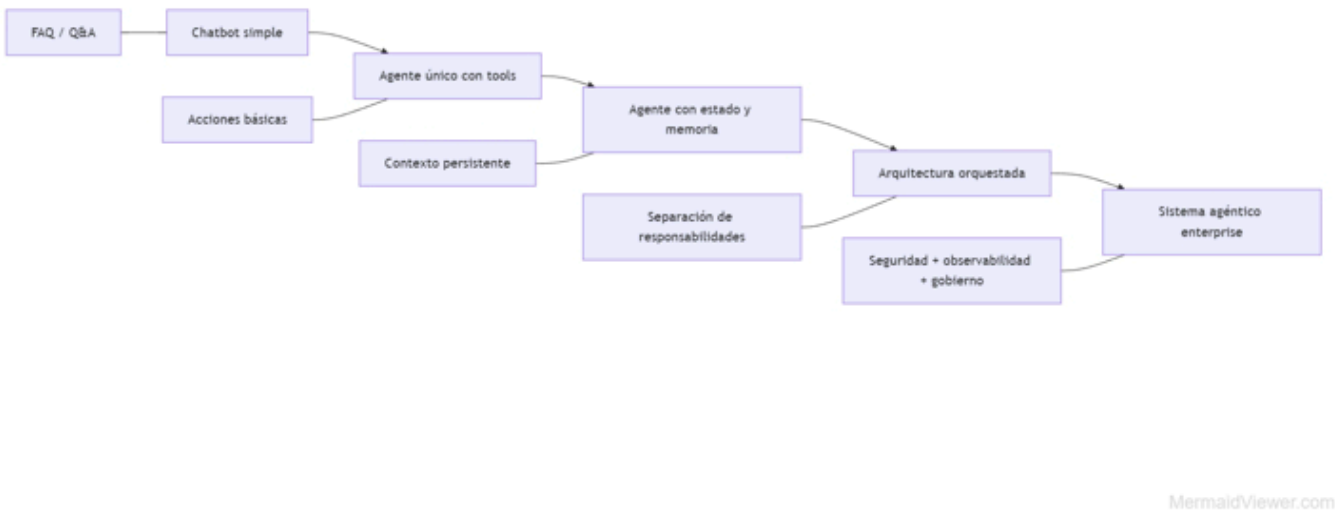
- interpretar intención,
- decidir,
- consultar datos,
- invocar APIs,
- gestionar memoria,
- redactar respuesta,
- controlar errores.

Eso genera varios problemas:

- prompts demasiado grandes,
- dificultad para depurar,
- poca explicabilidad,
- fragilidad ante cambios,
- acoplamiento fuerte,
- y riesgo operativo.

La evolución natural consiste en:

1. **chatbot simple**, a
2. **agente único con tools**, a
3. **agente con estado y memoria**, a
4. **arquitectura multiagente u orquestada**, a
5. **plataforma enterprise con evaluación, guardrails y gobierno**.



Qué entendemos por sistema agéntico enterprise

Es una arquitectura en la que cada componente tiene un papel claro:

- **Azure OpenAI** aporta la capa de inferencia y despliegue de modelos.
- **Foundry** aporta plataforma para desarrollo, evaluación, operación y capacidades gestionadas.
- **MAF** aporta una base más modular y controlada para construir agentes y workflows.
- **Copilot Studio** puede actuar como capa de experiencia cuando lo que importa es la interfaz y la integración con negocio.

Conceptos

Chatbot simple

Sistema conversacional orientado a responder preguntas o mantener diálogo básico.
Suele ser suficiente para:

- FAQ,
- asistencia interna muy acotada,
- demos,
- prototipos rápidos.

Agente aislado

Un único agente con cierto grado de autonomía, capaz de:

- razonar,
- elegir herramientas,
- y producir una respuesta más útil.

Sistema agéntico

Conjunto de componentes coordinados que combinan:

- LLM,
- herramientas,
- memoria,
- datos,
- reglas,
- workflow,
- y observabilidad.

Orquestación

Mecanismo que decide:

- qué paso se lanza,
- en qué orden,
- con qué datos,
- con qué política,
- y con qué tratamiento de error.

Tools

Capacidades externas invocables por el agente:

- consultar CRM,
- crear ticket,
- leer inventario,
- buscar documentos,
- actualizar pedido,
- enviar aprobación.

Memoria

Persistencia útil del contexto a corto o largo plazo.

Ejemplos:

- preferencias del usuario,
- estado del proceso en curso,
- decisiones ya tomadas,
- histórico relevante,
- resultados intermedios.

Observabilidad

Capacidad para saber:

- qué hizo el sistema,
- qué tool invocó,
- con qué contexto,
- por qué falló,
- cuánto costó,
- cuánto tardó,
- y qué respuesta devolvió.

Gobierno

Conjunto de controles para que el sistema sea aceptable en producción:

- identidad,
- permisos,
- secretos,
- políticas,
- evaluación,
- guardrails,
- auditoría,
- versionado,
- coste y cumplimiento.

> **Un chatbot responde.**
> **Un agente actúa.**
> **Un sistema agéntico enterprise coordina, controla, observa y gobierna.**

Ejercicio. Diagnóstico de madurez

Dados tres casos:

- FAQ de RRHH,
- asistente de soporte IT,
- agente de operaciones que consulta ERP y crea incidencias.

Cuando usarías chatbot simple, agente único, sistema orquestado.

Limitaciones de enfoques simples cuando aparecen tools, memoria, datos y operación a escala

Un sistema sencillo puede parecer correcto en una demo porque:

- responde bien,
- usa una tool,
- y mantiene una conversación razonable.

Pero cuando aparecen **tools reales, estado, permisos, datos sensibles y procesos largos**, el problema ya no es “montar una buena respuesta”, sino **controlar una arquitectura**.

> **La limitación de los enfoques simples no está solo en la calidad del texto de la respuesta. Está en su incapacidad para ser fiables, gobernables y mantenibles.**

Por qué algo simple puede parecer suficiente

Un chatbot simple o un agente pequeño suele funcionar bien en escenarios como:

- el usuario pregunta,
- el modelo responde,
- quizá llama a una API,

- y devuelve una salida correcta.

Señales de que debes evolucionar más allá del chatbot

1. El agente debe invocar más de dos o tres herramientas.
2. El proceso tiene varios pasos o estados.
3. Hay aprobaciones humanas.
4. Hay datos sensibles o segmentados por rol.
5. El cliente pide auditoría o explicación.
6. El coste y la latencia importan de verdad.
7. Debes reutilizar componentes entre varios casos de uso.
8. Debes operar en producción con soporte real.

Qué cambia cuando aparecen tools, memoria y datos

En cuanto añadimos estos elementos, cambian las reglas del juego.

Con tools

El sistema deja de ser solo conversacional, ahora también **actúa**.

Eso implica:

- riesgo operativo,
- contratos de entrada y salida,
- control de errores,
- autenticación,
- trazabilidad,
- impacto en negocio.

Con memoria

El sistema deja de vivir en el turno actual.
Ahora debe decidir:

- qué recordar,
- durante cuánto tiempo,
- con qué propósito,

- y con qué nivel de seguridad.

Con datos corporativos

El sistema deja de trabajar con contexto “de ejemplo”.
Ahora entra en juego:

- confidencialidad,
- minimización de dato,
- segmentación por rol,
- aislamiento por cliente o dominio,
- grounding,
- versionado,
- cumplimiento.

Con operación a escala

El sistema deja de ser una demo.
Ahora importa:

- latencia,
- concurrencia,
- coste,
- reintentos,
- observabilidad,
- soporte,
- despliegue,
- evolución y gobierno.

Conceptos clave

> **Agente monolítico**
>
> Un solo agente concentra toda la lógica de interpretación, decisión y acción.

Grounding

Uso de datos o documentos como soporte factual de la respuesta.

Cuándo LangGraph encaja y cuándo no según control, complejidad y necesidades de producción

LangGraph **encaja muy bien** cuando el problema central es la **orquestación explícita**:

- estados,
- nodos,
- transiciones,
- rutas de decisión,
- human-in-the-loop,
- control fino del flujo.

Pero LangGraph **no es una plataforma enterprise completa por sí sola**.
Es, sobre todo, un **framework de control y composición**.

Eso significa que brilla cuando quieres **construir tú la arquitectura**, pero pierde ventaja cuando necesitas que muchas capacidades lleguen **ya resueltas o más gestionadas**

Cuándo encaja bien

LangGraph encaja bien cuando la prioridad es **tener control explícito de la lógica**.

Caso 1. Workflows con pasos no triviales

Caso 2. Necesitas estado y checkpointing

Caso 3. Quieres control fino sobre la orquestación

Caso 4. Equipos técnicos que ya vienen de LangChain

Caso 5. Prototipos avanzados con arquitectura seria

Cuándo no encaja tan bien

LangGraph empieza a encajar peor cuando el problema principal ya no es la orquestación, sino la **industrialización**.

1. Cuando quieres capacidades enterprise gestionadas de serie

Si lo que más te importa es disponer con menos esfuerzo de:

- identidad,
- observabilidad integrada,
- evaluación,
- operación,
- persistencia útil gestionada,
- y controles más nativos,

LangGraph por sí solo no resuelve eso.

2. Cuando el equipo no quiere construir tanta infraestructura alrededor

LangGraph da libertad, pero esa libertad implica trabajo.

3. Cuando el caso de uso es más estándar que complejo

4. Cuando hay fuertes exigencias de producción y soporte

Opciones

- **LangGraph solo** para control máximo y casos muy técnicos.
- **LangGraph + Azure OpenAI** para inferencia y orquestación propia.
- **LangGraph + Foundry** cuando quieres conservar control técnico pero ganar capacidades gestionadas.
- **MAF o Foundry** cuando priorizas robustez, modularidad, observabilidad y operación enterprise.
- **Copilot Studio sobre backend avanzado** cuando además necesitas capa de experiencia de negocio.

Conceptos clave

LangGraph

Framework orientado a modelar agentes y workflows como un grafo de:

- nodos,
- estado,
- transiciones,
- y decisiones.

Control explícito

Capacidad de definir de forma clara:

- qué paso ocurre,
- cuándo,
- con qué estado,
- y con qué rutas alternativas.

Checkpointing

Persistencia intermedia para poder:

- reanudar,
- inspeccionar,
- o controlar procesos largos.

Human-in-the-loop

Patrón en el que una persona interviene en un punto concreto para:

- aprobar,
- corregir,
- o desbloquear una ejecución.

Framework vs plataforma

Un **framework** te da piezas para construir.
Una **plataforma** te da además parte de la operación, gobierno y capacidades gestionadas.

> Si tu principal problema es modelar bien el flujo, LangGraph encaja.
> Si tu principal problema es operar, gobernar y escalar el sistema, LangGraph solo ya no basta.

Ejercicio. Clasificar casos

Dedos estos escenarios, decidir si LangGraph encaja mucho, poco o nada:

- FAQ con RAG básico,
- agente de compras con aprobación,
- sistema multiagente con handoffs,
- asistente simple para consulta de horario,
- proceso de soporte con varias rutas y reintentos.

Cuándo usar Foundry frente a frameworks puros para ganar capacidades gestionadas y gobierno

Un **framework puro** te da mucha libertad para construir.
Foundry te da, además, una parte importante del entorno para **desarrollar, evaluar y operar** agentes con más capacidades gestionadas.

La diferencia real está en:

- ¿Cuánto quieres construir tú mismo?
- ¿Cuánto quieres que la plataforma te dé ya resuelto?
- ¿Cuánto pesan la seguridad, el gobierno y la operación?
- ¿Cuánto riesgo tienes si la solución falla, deriva o no deja trazabilidad?

Porque en la práctica:

- un framework puro puede seguir existiendo dentro de una solución en Azure,

- Foundry puede convivir con LangChain o LangGraph,
- y muchas arquitecturas reales son híbridas.

Cuándo un framework puro sigue encajando bien

1. Cuando la prioridad es control técnico absoluto

2. Cuando el caso está aún en fase exploratoria

3. Cuando el equipo ya tiene mucha infraestructura propia

4. Cuando el problema principal es la lógica del flujo

Cuándo Foundry empieza a tener más sentido

1. Cuando necesitas capacidades gestionadas

Foundry aporta valor cuando quieres reducir el trabajo de montar a mano piezas como:

- observabilidad,
- tracing,
- evaluación,
- memoria útil,
- identidad,
- operación,
- y ciertos aspectos de gobierno.

2. Cuando el cliente pide gobierno

3. Cuando hay varios equipos o varios proyectos

4. Cuando el time-to-production importa

5. Cuando la solución va a vivir en Azure y en contexto Microsoft

> Si tu problema principal es construir algo muy a medida, frameworks puros tienen mucho sentido.
> Si tu problema principal es industrializar, observar, evaluar y gobernar, Foundry gana mucha fuerza.

Conceptos clave

Framework puro

Conjunto de librerías y patrones para construir agentes y flujos con máxima flexibilidad, pero sin que la plataforma te resuelva por defecto muchas capas de operación.

Ejemplos conceptuales:

- composición,
- estado,
- tools,
- routing,
- memoria,
- workflows.

Plataforma gestionada

Entorno que no solo permite desarrollar, sino también aportar capacidades más listas para usar en:

- evaluación,
- observabilidad,
- identidad,
- memoria,
- seguridad,
- gobierno,
- y operación.

La decisión híbrida suele ser la más realista

- usar **frameworks** donde importa el control del flujo,
- y usar **Foundry** donde importa ganar:
  - observabilidad,
  - evaluación,
  - identidad,

- operación,
- y gobierno.

Rol de Azure OpenAI como capa de modelos y despliegues dentro de la arquitectura

Azure OpenAI es, sobre todo, la **capa de acceso a modelos** dentro de la solución.
Es la pieza que aporta:

- inferencia,
- despliegues de modelos,
- embeddings,
- configuración de consumo,
- controles de acceso en entorno Azure,
- y una base robusta para integrar capacidades LLM dentro de una arquitectura mayor.

> **Azure OpenAI pone la inteligencia model-driven**.
> La arquitectura completa la ponen además la orquestación, las tools, los datos, la observabilidad, la seguridad y el gobierno.

Qué lugar ocupa Azure OpenAI en la arquitectura

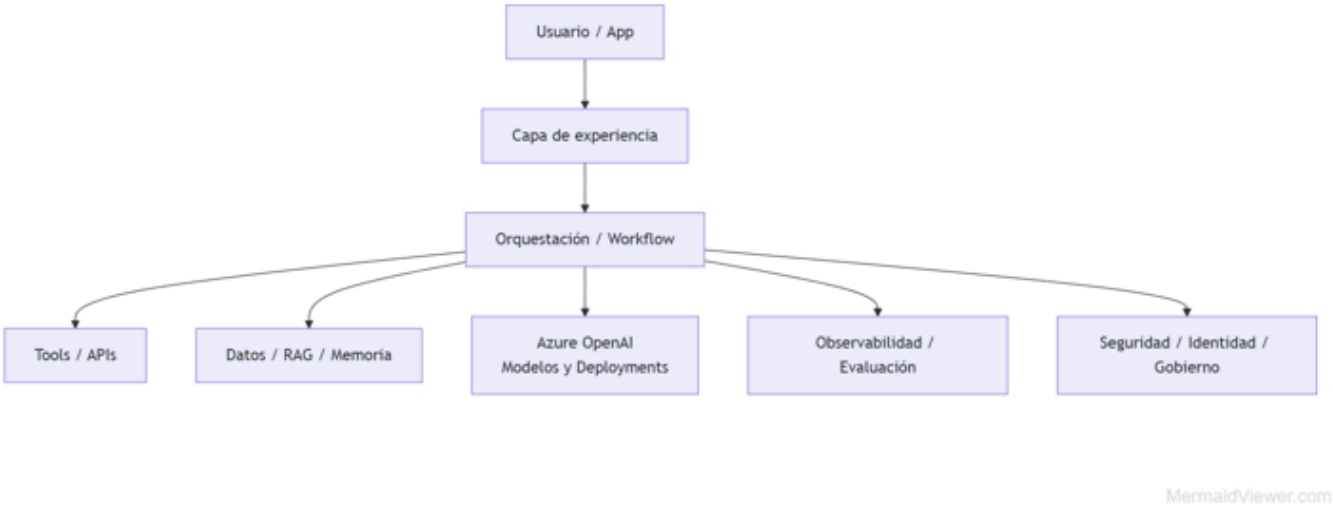
Cuando diseñamos una solución de agentes, suelen aparecer varias capas:

- experiencia,
- orquestación,
- tools,
- datos,
- memoria,
- observabilidad,
- seguridad,
- y modelos.

Azure OpenAI se sitúa principalmente en esta capa: la **capa de modelos e inferencia**.

Eso significa que Azure OpenAI:

- genera texto,
- interpreta instrucciones,
- hace function calling,
- genera embeddings,
- ayuda a clasificar, resumir, extraer o transformar,
- y puede ser el núcleo conversacional del agente.



Qué decisiones arquitectónicas dependen de Azure OpenAI

Decisión 1. Qué modelo usar según la tarea

Decisión 2. Cómo organizar deployments

Decisión 3. Cómo actualizar versiones sin romper comportamiento

Decisión 4. Qué parámetros de inferencia usar

Decisión 5. Cómo diseñar coste y latencia desde el inicio

Conceptos clave

Azure OpenAI

Servicio que permite consumir modelos en entorno Azure como base de inferencia para aplicaciones y agentes.

Inferencia

Proceso de enviar una entrada al modelo y recibir una salida:

- texto,
- estructura,
- clasificación,
- embeddings,
- decisión guiada.

Deployment

Instancia operativa y direccionable de un modelo dentro del recurso Azure OpenAI.

Es una idea clave porque el código de aplicación suele llamar a un **deployment**, no al “modelo abstracto” de forma directa.

Endpoint

Punto de acceso del servicio al que la aplicación se conecta para consumir inferencia o embeddings.

Embeddings

Representaciones vectoriales útiles para:

- similitud semántica,
- búsqueda,
- clustering,
- grounding,
- recuperación documental.

Parámetros de inferencia

Configuración que influye en el comportamiento del modelo:

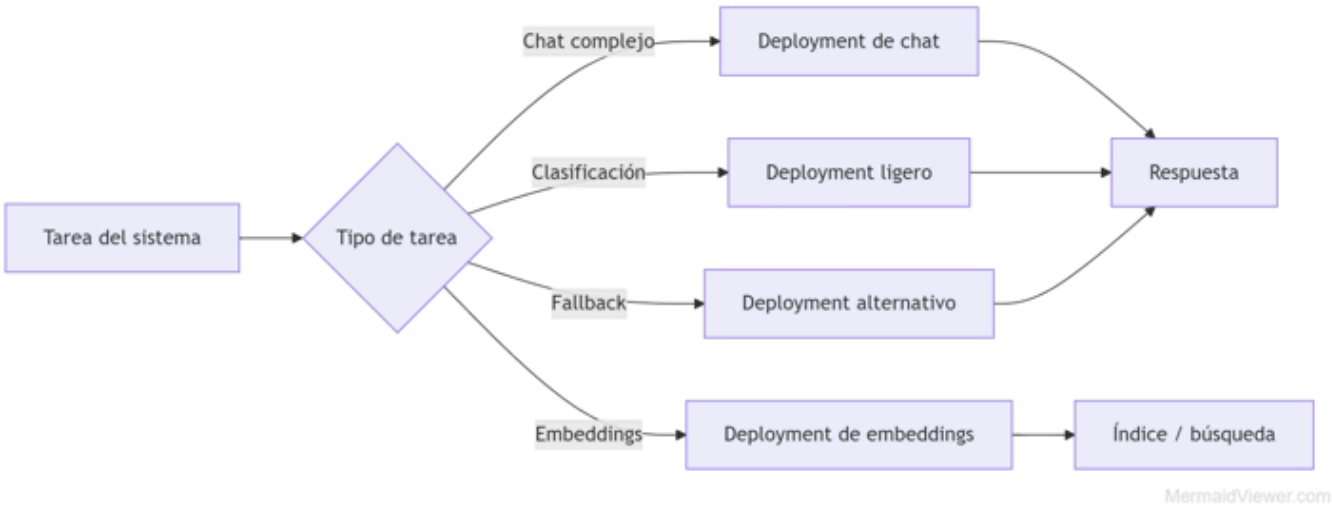
- temperatura,
- top_p,
- longitud máxima,
- etc.

Routing de modelos

Decisión arquitectónica que determina qué deployment usar según:

- tipo de tarea,
- criticidad,
- coste,
- latencia,
- o fallback.

Azure OpenAI como capa de decisión de modelo



Setup Operativo

```
$env:AZURE_OPENAI_ENDPOINT="https://curso-ia-openai.openai.azure.com/"
$env:AZURE_OPENAI_ENDPOINT_RESP="https://curso-ia-openai-
responses.cognitiveservices.azure.com/"
pip install --upgrade openai azure-identity
az login

git clone dmonto/curso-azureai
cd curso-azureai
python -m venv .venv
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.venv\Scripts\Activate.ps1
```

Ejemplo: llamada básica a ChatCompletions

az101.py

```
import os
from azure.identity import DefaultAzureCredential, get_bearer_token_provider
from openai import AzureOpenAI

token_provider = get_bearer_token_provider(
    DefaultAzureCredential(),
    "https://cognitiveservices.azure.com/.default"
)

client = AzureOpenAI(
    azure_endpoint=os.getenv("AZURE_OPENAI_ENDPOINT"),
```

Ejemplo: llamada básica a Responses

az101b.py

```
import os
from azure.identity import DefaultAzureCredential, get_bearer_token_provider
from openai import OpenAI

endpoint = os.getenv("AZURE_OPENAI_ENDPOINT_RESP")

if not endpoint:
    raise RuntimeError("Falta la variable AZURE_OPENAI_ENDPOINT_RESP")

# Normaliza endpoint por si viene con / final
base_url = endpoint.rstrip("/") + "/openai/v1/"
```

Ejemplo: routing entre deployments según tarea

az102.py

```
import os
from azure.identity import DefaultAzureCredential, get_bearer_token_provider
from openai import AzureOpenAI

token_provider = get_bearer_token_provider(
    DefaultAzureCredential(),
    "https://cognitiveservices.azure.com/.default"
)

client = AzureOpenAI(
    azure_endpoint=os.getenv("AZURE_OPE[illegible]NDPOINT"),
```

Ejemplo: embeddings como otra función de la capa Azure OpenAI

az103.py

```
import os
from azure.identity import DefaultAzureCredential, get_bearer_token_provider
from openai import AzureOpenAI

token_provider = get_bearer_token_provider(
    DefaultAzureCredential(),
    "https://cognitiveservices.azure.com/.default"
)

client = AzureOpenAI(
    azure_endpoint=os.getenv("AZURE_OPENAI_ENDPOINT"),
```

Buenas prácticas

1. Tratar Azure OpenAI como una capa, no como toda la solución

2. Nombrar deployments con criterio

3. Separar deployments por uso

4. Probar cambios de versión antes de promoverlos

5. Ajustar parámetros según caso

6. Monitorizar coste y latencia por flujo

7. Encapsular el acceso al servicio

8. Integrarlo con identidad, secretos y gobierno

Ejercicio. Mini-Chatbot

Partiendo de los scripts anteriores, crear uno nuevo que:

- Acepte input del usuario
- Tenga un bucle (con 's' para salir)
- Use los 3 deployments
- En caso de embeddings, muestre parte de [illegible]enido

Rol de Microsoft Foundry como plataforma de desarrollo, evaluación y operación de agentes

> **Foundry actúa como plataforma de desarrollo, evaluación y operación de agentes.**

Foundry aporta:

- projects,
- agents,
- tools,
- knowledge,
- evaluación,
- tracing,
- observabilidad,
- memory,
- identidad,
- y gobierno.

Un framework te ayuda a construir lógica

Por ejemplo:

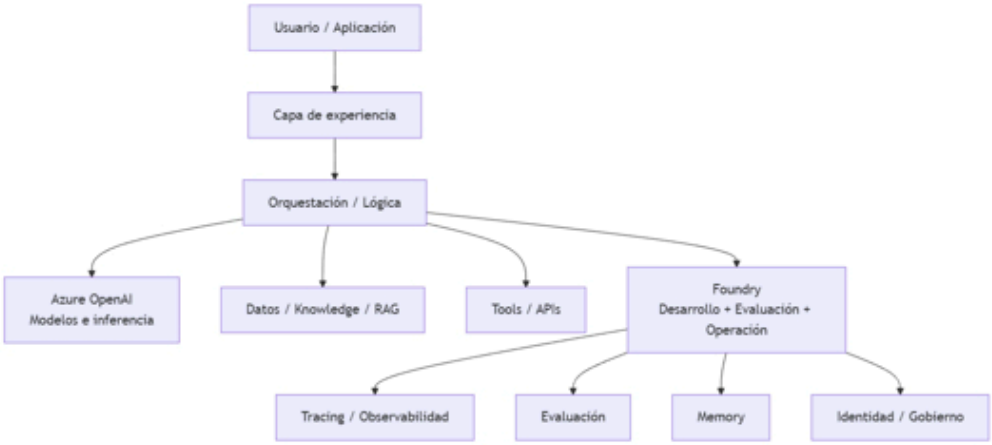
- nodos,
- rutas,
- tools,
- estado,
- workflow.

Una plataforma te ayuda además a operar esa lógica

Por ejemplo:

- organizar proyectos,
- centralizar capacidades,

- observar ejecuciones,
- evaluar resultados,
- conectar identidad,
- gobernar datos,
- y dar soporte a la evolución del sistema.



Conceptos clave

Platform thinking

Forma de pensar en la que no solo importa construir lógica, sino también:

- cómo se desarrolla,
- cómo se observa,
- cómo se evalúa,
- cómo se gobierna,
- y cómo se opera.

Project

Unidad de organización de trabajo dentro de la plataforma.

Sirve para que el desarrollo no sea una colec[illegible]spersa de scripts o pruebas aisladas.

Tool

Capacidad conectada al agente para consultar o actuar sobre sistemas externos.

Knowledge

Capa de conocimiento o grounding sobre la que el agente puede apoyarse.

Tracing

Capacidad para seguir la ejecución de una petición paso a paso.

Evaluation

Proceso sistemático para medir si el agente cumple calidad, seguridad y comportamiento esperado.

Rol de Microsoft Agent Framework como base para construir soluciones más controladas y modulares

MAF es un framework para construir agentes y workflows multiagente en **Python y .NET**, sucesor directo de **AutoGen** y **Semantic Kernel**, combinando abstracciones sencillas de agentes con capacidades más enterprise como **state management por sesión, type safety, middleware, telemetry** y **workflows** para control explícito de ejecución.

- **Azure OpenAI** pone la inferencia.
- **Foundry** pone la plataforma gestionada.
- **MAF** pone una base de construcción para que el sistema esté mejor separado, más controlado y sea más mantenible.

Incluye dos grandes capacidades: **agents**, que usan LLMs, tools y servidores MCP para responder, y **workflows**, que conectan agentes y funciones con routing tipado, checkpointing y human-in-the-loop

En el Modelo, MAF vive en el nivel de **framework**. No es el modelo y tampoco es la plataforma fully managed. Su papel es dar **abstracciones consistentes** para que una solución de agentes no sea una suma caótica de llamadas al LLM, tools y estado disperso.

Qué significa “mayor control” en MAF

MAF introduce workflows basados en grafo con **control explícito sobre las rutas de ejecución**, además de soporte para escenarios **long-running** y **human-in-the-loop**.

También distingue claramente entre usar un **agent** cuando la tarea es abierta o conversacional y usar un **workflow** cuando hay pasos bien definidos y necesitas control explícito del orden de ejecución.

- si la tarea necesita autonomía limitada, un agente puede bastar;
- si la tarea necesita pasos, validación, branching o coordinación entre componentes, MAF permite modelarlo de forma más explícita.

Además, los workflows validan compatibilidad de tipos, conectividad, binding de ejecutores y edges, lo que reduce errores de integración que en enfoques más artesanales suelen aparecer tarde, en runtime.

Qué significa “más modularidad” en MAF

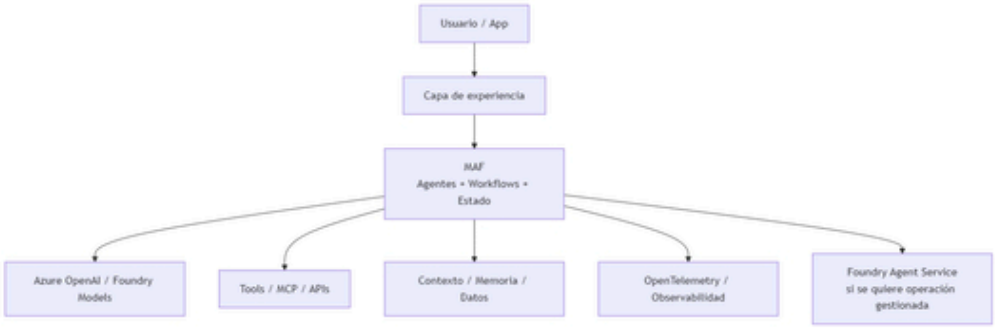
La modularidad en MAF no consiste solo en “tener varias clases”. Consiste en separar responsabilidades:

- agente o agentes,
- tools,
- estado,
- contexto,
- memoria,
- workflow,
- observabilidad.

El framework tiene soporte para **tools decoradas**, **context providers**, **session state**, distintos providers de agente y capas de workflow. Eso permite construir componentes intercambiables y más mantenibles que un agente monolítico.

Qué relación tiene con Foundry

- **MAF** es el framework con el que construyes la lógica controlada.
- **Foundry Agent Service** es la plataforma gestionada que puede hospedar agentes construidos con Agent Framework, LangGraph o incluso código propio, y encargarse de hosting, scaling, identidad, observabilidad y seguridad enterprise.



MermaidViewer.com

Conceptos clave

Framework base

MAF debe verse como la base de construcción para agentes y workflows, no como el modelo ni como la plataforma gestionada.

Agent

Un agent usa LLM, tools y contexto para responder o actuar. Es apropiado cuando la tarea es abierta o conversacional.

Workflow

Es la pieza adecuada cuando el proceso tiene pasos definidos, coordinación entre funciones o agentes, branching y control explícito.

Type safety

MAF enfatiza type safety y validación para que el paso de mensajes entre componentes sea más robusto. La documentación de workflows habla explícitamente de compatibilidad de tipos y validaciones estructurales.

Session-based state

La documentación de overview sitúa la gestión de estado por sesión como una de las capacidades heredadas del legado enterprise de Semantic Kernel.

Ejemplo: agente mínimo con MAF

```
pip install agent_framework
```

az104.py

```
import asyncio

from agent_framework import Agent
from agent_framework.openai import OpenAIChatCompletionClient
from azure.identity.aio import AzureCliCredential


async def main() -> None:
    credential = AzureCliCredential()
    try:
        client = OpenAIChatCompletionClient(
```

Ejemplo: tool con contrato

az105.py

```
import asyncio
from typing import Annotated

from agent_framework import Agent, tool
from agent_framework.openai import OpenAIChatCompletionClient
from pydantic import Field
from azure.identity.aio import AzureCliCredential


@tool(approval_mode="never_require")
def get_weather(
```

Cuándo MAF aporta mucho valor

1. Cuando quieres control explícito sin bajar a “todo a mano”

2. Cuando el sistema ya no cabe bien en un solo agente

3. Cuando importa la robustez de integración

4. Cuando quieres observabilidad desde el diseño

Cuándo MAF no basta por sí solo

1. Cuando necesitas una plataforma fully managed

2. Cuando el problema es muy simple

Ejercicio. Extender Tools

Extender ejemplo con otra tool, con:

- tipos,
- validación,
- descripción,
- y política de aprobación.

Relación con Copilot Studio cuando se necesita una capa de experiencia sobre la arquitectura avanzada

Copilot Studio debe entenderse, sobre todo, como una herramienta **gráfica y low-code** para construir agentes y agent flows, conectar datos mediante conectores predefinidos o personalizados, y publicar experiencias en canales como sitios web, apps móviles, Facebook, Microsoft Teams o canales compatibles con Azure Bot Service.

Cuando una organización ya tiene una arquitectura avanzada con **Azure OpenAI**, **Foundry**, **MAF**, tools, RAG, observabilidad y gobierno, **Copilot Studio no suele sustituir esa base**.

Lo que hace bien es poner encima una **capa de interacción, canales, experiencia conversacional y negocio**. Esa lectura encaja con la propia arquitectura de agentes de Microsoft, donde el **cliente/interfaz** es una capa distinta del orquestador, del modelo y del tool calling.

> **Foundry/MAF construyen el cerebro operativo.**
> **Copilot Studio acerca ese cerebro al usuario de negocio.**

Qué significa “capa de experiencia”

Copilot Studio encaja muy bien en la parte de:

- **cliente/interfaz**,
- **experiencia conversacional**,
- **canales**,
- **configuración low-code de temas, tools y conectores**,
- y cierta orquestación de negocio orientada a experiencia.

Conviene diferenciar los **custom engine agents**, que dan control total sobre orquestación, modelos e integraciones de datos; esos agentes pueden construirse con enfoque low-code mediante Copilot Studio, con enfoque pro-code, o integrando un agente existente de Microsoft Foundry en Microsoft 365.

Cuándo Copilot Studio encaja muy bien

1. Cuando el usuario final vive en Teams o Microsoft 365

Foundry permite publicar agentes a Microsoft 365 Copilot y Teams mediante una agent application con endpoint estable, orientada precisamente a consumo en esas interfaces.

2. Cuando importa la experiencia más que el backend

Hay proyectos donde la dificultad no está tanto en el modelo o el workflow, sino en:

- cómo guiar la conversación,
- cómo conectar topics,
- cómo exponer tools,
- cómo trabajar con knowledge,
- cómo hacer que negocio pueda iterar.

En esos casos, Copilot Studio aporta mucho porque permite construir y ajustar experiencia conversacional sin bajar siempre al nivel de framework puro.

3. Cuando quieres conectar tools, knowledge y otros agentes desde una capa low-code

Con **generative orchestration**, Copilot Studio puede seleccionar uno o más tools, topics, otros agentes o knowledge sources para responder a una consulta, incluso en consultas multi-intent o en respuestas autónomas a eventos. Esa selección depende sobre todo de la **descripción** de los elementos conectados, además de sus nombres y parámetros.

4. Cuando quieres componer sobre un backend más avanzado

Copilot Studio puede conectarse a un **Microsoft Foundry agent** como agente externo, usando la conexión al project endpoint y el agent id. Microsoft indica además que la descripción del agente conectado es clave para que el agente principal sepa cuándo debe usarlo.

5. Cuando necesitas knowledge corporativo desde una experiencia unificada

Copilot Studio hace trabajar las knowledge sources junto con las generative answers, y esas fuentes pueden venir de Power Platform, Dynamics 365, websites y sistemas externos. También soporta fuentes custom mediante `OnKnowledgeRequested`, lo que permite usar un search endpoint propio, Azure AI Search u otros mecanismos bajo control de la organización.

Cuándo Copilot Studio no debería ser “el centro técnico”

1. Cuando el valor principal está en la orquestación profunda

2. Cuando el backend necesita pro-code y g[illegible]no más fino

Conceptos clave

Capa de experiencia

Es la parte del sistema con la que interactúa el usuario:

- chat,
- canales,
- topics,
- mensajes,
- handoff conversacional,
- experiencia contextual.

Generative orchestration

Modo en el que Copilot Studio puede seleccionar tools, topics, otros agentes y knowledge sources, incluso varios en secuencia, usando descripciones y contexto conversacional para decidir.

Connected agent

Agente externo conectado al agente principal de Copilot Studio. Puede ser, por ejemplo, un Microsoft Foundry agent.

Knowledge source

Fuente de conocimiento usada por el agente para grounding. Puede venir de sistemas Microsoft o de endpoints propios.

Custom engine agent

Agente de Microsoft 365 Copilot con control total sobre orquestación, modelos e integraciones, frente a experiencias más estándar o declarativas.

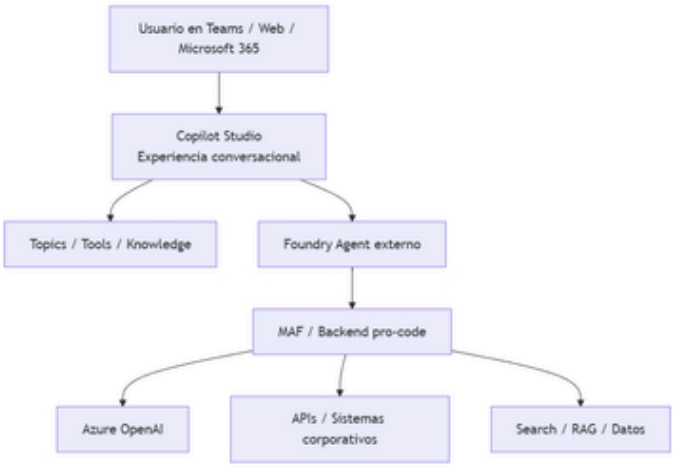
Ejemplo: backend de acción para Copilot Studio

```
from fastapi import FastAPI
from pydantic import BaseModel, Field
from typing import Literal

app = FastAPI(title="Support Action API")


class TriageRequest(BaseModel):
    user_id: str
    message: str = Field(..., min_length=3)
```

Visión integral del stack para que cada pieza tenga un lugar claro en la solución



MermaidViewer.com

Capa de experiencia y canales

Aquí vive la interacción con el usuario:

- Teams,
- web,
- Microsoft 365,

- chat,
- experiencia conversacional,
- tópicos, flujos y UX de negocio.

Copilot Studio encaja aquí: una herramienta **gráfica y low-code** para construir agentes y agent flows, con conectores predefinidos o personalizados y publicación en múltiples canales, incluidos Teams y otros compatibles con Azure Bot Service.

Capa de tools y acciones

Aquí están:

- APIs corporativas,
- ERP,
- CRM,
- bases de datos,
- acciones transaccionales,
- integraciones externas.

Capa de observabilidad, evaluación y guardrails

Aquí vive lo que hace operable al sistema:

- tracing,
- métricas,
- calidad,
- seguridad,
- evaluación funcional,
- evaluación de tool calling,
- safety,
- mejora continua.

Enfoque del curso como puente entre open source, servicios Azure y operación enterprise

Muchos equipos técnicos están especializados:

- saben construir cosas con librerías open source,
- saben consumir servicios de Azure,
- o saben operar entornos enterprise con requisitos de seguridad y control.

Lo que suele faltar es la **conexión entre esos tres planos**.

Ese gap es el que cubre el curso. No se limita a “usar Foundry” o “usar Azure OpenAI”, sino a enseñar **cómo pasar de un prototipo técnico a una solución operable**.

Foundry es la pieza clave, como plataforma para **construir, customizar, evaluar y operar** agentes y los modelos y tools que usan; Foundry Agent Service, además, es una plataforma totalmente gestionada para construir, desplegar y escalar agentes, incluso si están hechos con Agent Framework, LangGraph o código propio.

Los tres mundos que el curso conecta

1. Mundo open source

Aquí viven:

- LangChain,
- LangGraph,
- patrones de agent orchestration,
- control fino del flujo,
- prototipos rápidos,
- libertad de diseño.

2. Mundo Azure

Aquí viven:

- Azure OpenAI como capa de inferencia y deployments,

- Azure AI Search como capa de recuperación,
- Key Vault y RBAC como capas de identidad y secreto,
- Foundry como plataforma de proyectos, modelos, agentes y monitorización.

3. Mundo enterprise

Aquí importan preguntas distintas:

- ¿cómo lo opero?
- ¿cómo lo observo?
- ¿cómo lo evalúo?
- ¿cómo lo securizo?
- ¿cómo justifico coste, calidad y comportamiento?
- ¿cómo lo mantengo cuando deja de ser una demo?

Qué suele pasar cuando esos mundos no están conectados

Cuando un equipo viene solo del mundo **open source**, suele tener:

- mucho control,
- mucha flexibilidad,
- pero más bricolaje operativo.

Cuando un equipo viene solo del mundo **Azure**, puede conocer bien recursos y permisos, pero no siempre domina:

- patrones avanzados de agentes,
- grafo, estado y routing,
- multiagente,
- handoffs,
- memoria,
- o tool orchestration.

Cuando un equipo viene solo del mundo **enterprise**, puede tener muy claro:

- seguridad,
- procesos,
- cumplimiento,

- observabilidad,
- gobierno,

pero no siempre sabe traducir eso a diseño agéntico moderno.

MICROSOFT FOUNDRY PARA EQUIPOS QUE VIENEN DE LANGCHAIN Y LANGGRAPH

Qué es Microsoft Foundry y qué problemas resuelve en una plataforma enterprise de agentes

Microsoft Foundry es una oferta **PaaS unificada de Azure** para operaciones de IA enterprise, construcción de modelos y desarrollo de aplicaciones.

Unifica **agentes, modelos y tools** bajo un único grupo de gestión, con capacidades enterprise integradas como **tracing, monitoring, evaluations, RBAC, networking y policies**.

El branding actual es **Microsoft Foundry**, evolución de lo que antes aparecía como Azure AI Studio / Azure AI Foundry.

Qué problema intenta resolver Foundry

El **Foundry Agent Service** es una plataforma **fully managed** para construir, desplegar y escalar agentes, y que se ocupa de **hosting, scaling, identity, observability y enterprise security**, de modo que el equipo pueda centrarse más en la lógica del agente.

Qué piezas introduce Foundry

1. Recurso Foundry + proyecto Foundry

Microsoft distingue claramente entre:

- **Foundry resource**: recurso Azure de nivel superior donde se gestionan gobierno, networking, seguridad y despliegues de modelos.
- **Project**: frontera de desarrollo dentro del recurso, donde los equipos construyen y evalúan casos de uso.
- **Project assets**: agentes, evaluaciones, archivos y artefactos relacionados.

2. Agents, tools, files y evaluaciones como activos de proyecto

Un proyecto Foundry sirve para organizar trabajo como **agents, evaluations y files** mientras se construyen aplicaciones con estado y se exploran ideas.

3. SDKs y endpoints unificados

Un recurso Foundry proporciona **acceso unificado a models, agents y tools**. Esto reduce fricción a nivel de desarrollo y evita que cada capacidad viva en una integración inconexa.

Qué problemas resuelve

Problema 1. Operación artesanal

Foundry introduce un **agent development lifecycle** explícito: crear, probar, añadir tools y datos, guardar versiones, depurar con tracing, evaluar calidad y seguridad, publicar un endpoint estable y monitorizar en producción.

Problema 2. Falta de observabilidad

Foundry Agent Service incluye **end-to-end tracing, metrics y Application Insights integration**, y Microsoft lo vende precisamente como forma de ver cada decisión del agente

Problema 3. Falta de evaluación sistemática

La evaluación es esencial antes del despliegue y permite fijar umbrales de aceptación. Además, Foundry ofrece evaluadores para **quality, safety y agent behavior**, y también evaluadores específicos para agentes, como **task adherence, intent resolution o tool call accuracy**.

Problema 4. Identidad y permisos pobres

Foundry separa **control plane** y **data plane**, cada uno con su superficie de autenticación y RBAC. Microsoft recomienda **Microsoft Entra ID** para producción por soporte de conditional access, managed identities y least-privilege RBAC, mientras que las API keys quedan más orientadas a evaluación rápida.

Problema 5. Memoria improvisada

Foundry Memory se define como una solución **managed, long-term memory** para continuidad entre sesiones, dispositivos y workflows. Microsoft además distingue explícitamente entre memoria de corto plazo, que suele vivir en el runtime del framework, y memoria de largo plazo, que Foundry gestiona como conocimiento persistente. Está en preview, así que conviene enseñarlo como capacidad potente pero todavía no completamente madura.

Problema 6. Integración pobre entre open source y plataforma

Microsoft documenta de forma oficial el uso de `langchain-azure-ai` como punto de entrada para aplicaciones LangChain y LangGraph con capacidades de Foundry, y también documenta trazas y memoria para esos mismos frameworks. Eso significa que Foundry no exige borrar la inversión previa del equipo; puede actuar como capa de industrialización sobre ella.

Conceptos clave

Microsoft Foundry

Oferta PaaS unificada de Azure para operaciones enterprise de IA, desarrollo de modelos y aplicaciones, con capacidades integradas de gestión, observabilidad, evaluación y gobierno.

Foundry resource

Recurso Azure de nivel superior donde se configuran seguridad, red, despliegues de modelos y gobierno general.

Foundry project

Marco de desarrollo dentro del recurso donde se organizan agentes, evaluaciones, archivos y otros activos.

Foundry Agent Service

Plataforma gestionada para construir, desplegar y escalar agentes, con hosting, scaling, identity, observability y enterprise security integrados.

Tool catalog

Catálogo desde el que se descubren, configuran y añaden tools a agentes o workflows.

Managed memory

Memoria de largo plazo gestionada por Foundry Agent Service para continuidad entre sesiones y personalización persistente. Está en preview

Agent lifecycle

Secuencia recomendada por Microsoft para construir, versionar, trazar, evaluar, publicar y monitorizar agentes.

Qué aporta sobre un stack open source puro en observabilidad, identidad, evaluación y operación

> Un stack open source puro te deja construir bien.
> Foundry te ayuda a **observar, securizar, evaluar y operar** mejor.

Observabilidad

La observabilidad suele ser uno de los mayores saltos de madurez.

Microsoft documenta que **Foundry Agent Service** incluye **end-to-end tracing, metrics y Application Insights integration**, y lo presenta explícitamente como forma de ver cada decisión que toma el agente. Además, el **Agent Monitoring Dashboard** permite seguir métricas operativas y resultados de evaluación en producción, incluyendo **token usage, latency, success rates y evaluation outcomes**.

Con Foundry, una parte relevante de ese trabajo ya tiene una vía oficial y homogénea. Además, el **Foundry Control Plane** permite observabilidad centralizada de agentes distribuidos en múltiples proyectos dentro de una suscripción, incluyendo inventario, estado, trazas, error rates, token usage y coste, siempre que haya Application Insights configurado.

Identidad

Foundry separa **control plane** y **data plane**, cada uno con su propia superficie de autenticación y RBAC. También soporta dos métodos de autenticación: **Microsoft Entra ID** y **API keys**, pero Microsoft recomienda Entra ID para producción porque aporta **conditional access, managed identities y granular RBAC**, mientras que las API keys sirven más para prototipos rápidos y carecen de trazabilidad por usuario.

Evaluación

Ejecutar evaluaciones dirigidas a agentes usando evaluadores integrados de **quality, safety y agent behavior**, y remarca que durante desarrollo conviene fijar **baselines** y **acceptance thresholds** antes de publicar. Además, el catálogo actual incluye evaluadores específicos para agentes como **task completion, task adherence, intent resolution, tool call accuracy** y **tool selection**.

Operativa

Es una plataforma **fully managed** para **building, deploying and scaling** agentes, y dice explícitamente que se encarga de **hosting, scaling, identity, observability y enterprise security** para que el equipo se centre en la lógica del agente.

El **Foundry Control Plane** permite gestión centralizada de inventario, salud y operaciones de ciclo de vida sobre agentes distribuidos en varios proyectos o incluso varias plataformas soportadas.

Integración oficial con LangChain y LangGraph para aprovechar inversión
técnica previa

`langchain-azure-ai` es el **punto de entrada** para construir aplicaciones LangChain y LangGraph con capacidades de Microsoft Foundry.

Ese paquete actúa como capa de integración para modelos, agents, tools, retrievers, vector stores, chat history, tracing y otros componentes de plataforma.

El paquete cubre entre otras:

- `langchain_azure_ai.agents` para Foundry Agent Service,
- `langchain_azure_ai.chat_models` para modelos de chat,
- `langchain_azure_ai.embeddings` para embeddings,
- `langchain_azure_ai.vectorstores` para Azure AI Search y Cosmos DB,
- `langchain_azure_ai.retrievers` para recuperación,
- `langchain_azure_ai.chat_message_histories` para historial/memoria,
- `langchain_azure_ai.tools` para tools como Document Intelligence o Logic Apps,
- y `langchain_azure_ai.callbacks` para tracing y OpenTelemetry.

Qué cambia al integrar Foundry

Cambio 1. El punto de conexión recomendado pasa a ser el project endpoint

Cambio 2. La configuración del agente puede centralizarse en Foundry

Cambio 3. La observabilidad se vuelve homogénea

Cambio 4. La memoria deja de ser solo runtime local

Cambio 5. Se incorporan capacidades de plat[illegible]a como middleware de seguridad

Conceptos clave

`langchain-azure-ai`

Paquete oficial que Microsoft documenta como punto de entrada para aplicaciones LangChain y LangGraph con capacidades de Foundry.

Project endpoint

Endpoint del proyecto Foundry que permite reutilizar autenticación empresarial con Entra ID y RBAC sobre clases compatibles del paquete.

`AgentServiceFactory`

Punto de inicio para componer nodos compatibles con LangGraph que se ejecutan a través de Foundry Agent Service.

`AzureAIOpenTelemetryTracer`

Tracer oficial para emitir spans OpenTelemetry desde LangChain/LangGraph a Azure Application Insights y Azure Monitor.

Foundry Memory

Memoria de largo plazo para preferencias y resúmenes persistentes; se recomienda mantener el estado inmediato en el runtime de LangChain o LangGraph.

Ejemplo: usar modelos de Foundry desde LangChain

```
python -m pip install "langchain>=1.2.13" "langchain-azure-ai==1.2.2" azure-
identity
$env:AZURE_OPENAI_DEPLOYMENT_1="gpt-4o"
$env:AZURE_OPENAI_ENDPOINT="https://cur[illegible]-openai.openai.azure.com/"
$env:AZURE_OPENAI_API_VERSION="2024-10-[illegible]
```

az106.py

```
import os
import sys

from azure.identity import DefaultAzureCredential, get_bearer_token_provider
from langchain_openai import AzureChatOpenAI


def require_env(name: str) -> str:
    value = os.getenv(name)
    if not value:
        raise RuntimeError(f"Falta la variable de entorno {name}")
```

Ejemplo: conectar LangGraph con Foundry Agent Service

```
import os

from azure.identity import DefaultAzureCredential
from langchain_core.messages import HumanMessage
from langchain_azure_ai.agents import AgentServiceFactory

factory = AgentServiceFactory(
    project_endpoint=os.environ["AZURE_AI_PROJECT_ENDPOINT"],
    credential=DefaultAzureCredential(),
)
```

Projects, agents, tools y knowledge como piezas base para construir soluciones reales

> Un **project** organiza.
> Un **agent** decide y actúa.
> Las **tools** conectan con capacidades externas.
> El **knowledge** aporta grounding y contexto útil.

La arquitectura distingue entre **Foundry resource**, **project** y **project assets**: el recurso superior concentra gobierno, seguridad, red y despliegues de modelos; el **project** es la frontera de desarrollo donde los equipos construyen y evalúan casos de uso; y los **project assets** incluyen archivos, agentes, evaluaciones y artefactos [illegible]onados.

Qué es un Project

El **project** es el **marco de desarrollo** dentro del Foundry resource: ahí es donde los equipos construyen y evalúan casos de uso, mientras el recurso superior concentra gobierno, seguridad, red y despliegues de modelos.

Los **project assets** son archivos, agentes, evaluaciones y artefactos relacionados, todos acotados a ese proyecto.

Primero, porque introduce una **unidad de trabajo real**. Ayuda a separar **gobierno** de **desarrollo**.

Qué es un Agent y por qué no equivale a "chatbot"

Un agente es una aplicación de IA que usa un LLM para razonar sobre peticiones del usuario y ejecutar acciones autónomas.

Un agente puede llamar tools, acceder a datos externos y tomar decisiones en varios pasos para completar una tarea.

Foundry Agent Service lo descompone en tres componentes básicos: **modelo**, **instrucciones** y **tools**.

Hoy, Agent Service soporta tres tipos principales de agentes:

- **prompt agents**, definidos por configuración y muy orientados a prototipos rápidos o tareas simples;
- **workflow agents**, para automatización multi-step, branching y coordinación más repetible;
- **hosted agents**, basados en código y pensados para máxima flexibilidad con frameworks como Agent Framework, LangGraph o código propio.

Qué son las Tools

Una tool es una capacidad que el agente puede invocar en una conversación para ejecutar una tarea específica: buscar en la web, correr código, consultar datos o llamar a tus propias APIs.

Foundry añade además un **tool catalog** como punto de descubrimiento y configuración

Muchas tools ya vienen con autenticación gestionada, incluidas credenciales administradas por el servicio y escenarios **On-Behalf-Of**.

Qué entendemos por Knowledge

Foundry IQ, es como una **managed knowledge layer** , capaz de conectar datos estructurados y no estructurados en Azure, SharePoint, OneLake y la web para dar a los agentes acceso a conocimiento **permission-aware**.

Foundry Agent Service también permite grounding mediante tools concretas como:

- **Azure AI Search tool**,
- file search,
- web search,
- y otras fuentes de datos del catálogo.

Cómo se combinan las cuatro piezas en una solución real

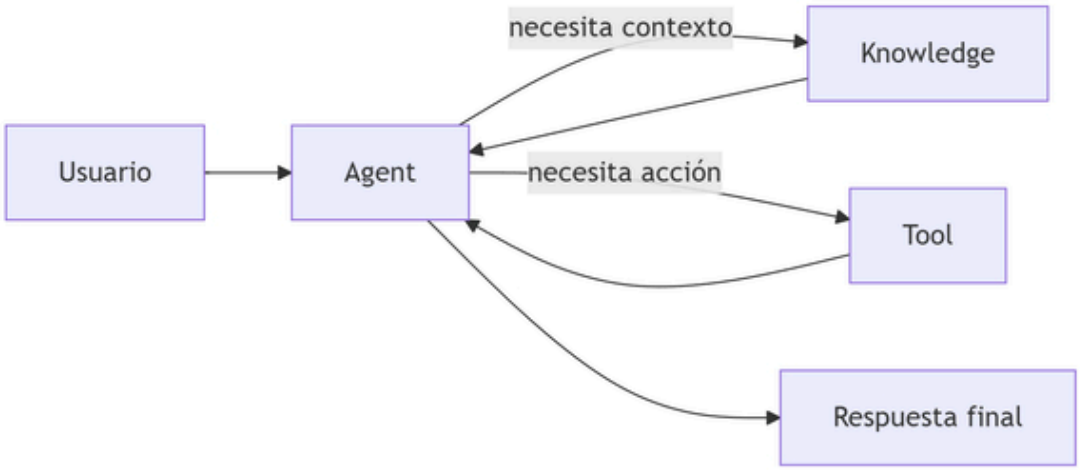
- el **project** delimita el caso de uso y sus activos,
- el **agent** encapsula la conducta y el tipo de orquestación,
- las **tools** conectan con acción y datos externos,
- el **knowledge** aporta grounding sobre información corporativa.

Conceptos clave

> **Project**
>
> Marco de desarrollo dentro de un Foundry resource. Organiza activos como agentes, evaluaciones y archivos, y separa trabajo del equipo de los controles centralizados del recurso.

> **Knowledge**
>
> Capa de grounding sobre conocimiento corporativo. Puede implementarse mediante Foundry IQ como knowledge layer gestionada, o mediante tools como Azure AI Search y file search.

Flujo de una petición real



Ejemplo: patrón de entrada a un Project en Foundry

az107.py

```
import os

from azure.identity import DefaultAzureCredential
from azure.ai.projects import AIProjectClient


PROJECT_ENDPOINT = os.getenv(
    "AZURE_AI_PROJECT_ENDPOINT",
    "https://curso-ia-openai.services.ai.azure.com/api/projects/curso-ia-openai-
project"
)
```

Evaluación, tracing y observabilidad para medir calidad, seguridad y comportamiento

En un prototipo, solemos validar el agente con una impresión subjetiva:

> “Le he hecho unas preguntas y parece que [illegible] bien”.

En producción, eso no es suficiente.

Microsoft Foundry plantea la observabilidad como una disciplina para medir y mejorar tanto la **calidad** como la **seguridad** de los outputs a lo largo de todo el ciclo de vida, y su dashboard de observabilidad se integra con **Azure Monitor Application Insights** para ofrecer visibilidad en tiempo real sobre rendimiento, seguridad y calidad.

> **Evaluación** responde a "¿lo hace bien?".
> **Tracing** responde a "¿cómo ha llegado hasta aquí?".
> **Observabilidad** responde a "¿qué está pasando en el sistema real, a escala y en el tiempo?".

El **Agent Monitoring Dashboard** sirve para seguir **token usage, latency, success rates y evaluation outcomes** en tráfico de producción, y la evaluación de agentes incluye dimensiones como **task completion, task adherence, intent resolution** y calidad del uso de tools.

Tracing

Tracing no es solo logging bonito. En Foundry, el tracing captura detalles de una ejecución del agente como:

- **inputs**,
- **outputs**,
- **tool usage**,
- **retries**,
- **latencies**,
- y **costs**.

Microsoft lo apoya sobre **OpenTelemetry** y almacena las trazas en **Azure Monitor Application Insights**.

Foundry también ofrece integraciones de tracing para frameworks populares, incluyendo **Microsoft Agent Framework**, **LangChain** y **LangGraph**, con cambios mínimos en código.

Observabilidad

La **observabilidad** va más allá de una traza individual. Se ocupa del comportamiento agregado del sistema en el tiempo.

El dashboard de observabilidad de Foundry, integrado con Application Insights, da visibilidad en tiempo real sobre:

- rendimiento,
- calidad,
- seguridad,
- y métricas operativas.

Y el **Agent Monitoring Dashboard** añade visión de producción sobre:

- uso de tokens,
- latencia,
- tasa de éxito,
- resultados de evaluación,
- siempre leyendo la telemetría de la instancia de **Application Insights** conectada al proyecto Foundry.

Además, el **Foundry Control Plane** permite seguimiento centralizado de salud, rendimiento, cumplimiento y eficiencia de coste a escala de flota, no solo de un agente aislado.

Evaluación

La **evaluación** sirve para medir si el agente cumple expectativas funcionales y de seguridad con criterios repetibles, no solo por intuición.

A. Evaluación del resultado

Entre los evaluadores documentados están:

- **Task Completion**
- **Task Adherence**
- **Task Navigation Efficiency**
- **Intent Resolution**
- y también evaluadores de **Relevance** y **Groundedness** para la respuesta final.

B. Evaluación del proceso

Aquí entran evaluadores como:

- **Tool Call Accuracy**

- **Tool Selection**
- **Tool Input Accuracy**
- **Tool Output Utilization**
- **Tool Call Success**.

Conceptos clave

Tracing

Captura detallada de una ejecución concreta del agente: inputs, outputs, tool usage, latencia, tokens, costes y errores.

Observabilidad

Conjunto de capacidades para entender el sistema en operación: trazas, métricas, dashboards, resultados de evaluación y señales agregadas en el tiempo

Evaluación

Proceso repetible para medir si el agente cumple calidad, seguridad y comportamiento esperados mediante datasets, umbrales y evaluadores definidos.

Application Insights

Backend de telemetría donde Foundry almacena trazas y desde el que se alimentan dashboards y análisis.

OpenTelemetry

Estándar sobre el que Foundry construye tracing e integración con frameworks y herramientas de monitorización.

Ejemplo: patrón de tracer oficial para LangChain/LangGraph

```
import os
from azure.identity import DefaultAzureCredential
from langchain_azure_ai.callbacks.tracers import AzureAIOpenTelemetryTracer

tracer = AzureAIOpenTelemetryTracer(
    project_endpoint=os.environ["AZURE_AI_PROJECT_ENDPOINT"],
    credential=DefaultAzureCredential(),
    name="support-agent-tracing",
    agent_id="support-agent",
    trace_all_langgraph_nodes=True,
    enable_content_recording=False,
```

Memory y capacidades gestionadas para dar persistencia útil sin montar toda la infraestructura a mano

Memory in Foundry Agent Service es una solución **managed, long-term memory** que permite continuidad del agente entre **sesiones, dispositivos y workflows**, reteniendo preferencias de usuario, historial relevante y experiencias personalizadas.

Esta capacidad está en **preview** y que su objetivo es la **memoria de largo plazo**, no sustituir todo el estado inmediato del framework.

> **Memoria no es almacenar conversaciones sin criterio.**
> **Memoria es persistir información útil, segura y recuperable para mejorar la continuidad del agente.**

Qué tipo de memoria resuelve Foundry y qué tipo no

1. Memoria de corto plazo

Es el contexto inmediato del flujo o de la conversación actual:

- los últimos mensajes,
- variables de trabajo,
- estado efímero del proceso,

- decisiones intermedias.

El patrón recomendado es **mantener el short-term chat history en el runtime** y usar **Foundry Memory como long-term store** para contexto a nivel de usuario.

2. Memoria de largo plazo

Es información más estable y destilada:

- preferencias del usuario,
- resúmenes útiles,
- hechos persistentes,
- contexto que debe sobrevivir a varias sesiones,
- continuidad entre dispositivos o workflows.

Esta memoria **extrae información significativa de conversaciones, la consolida en conocimiento duradero y la hace disponible a través de sesiones**.

Qué significa "capacidad gestionada" en este contexto

Los **memory stores** actúan como almacenamiento persistente y el parámetro **scope** sirve para segmentar la memoria entre usuarios y asegurar experiencias aisladas.

Cómo funciona Foundry Memory a alto nivel

1. **Extraction**: el sistema extrae preferencias, hechos y contexto relevante de la conversación.
2. **Consolidation**: usa lógica de consolidación para fusionar información similar o resolver duplicados/conflictos.
3. **Retrieval**: recupera los recuerdos más relevantes cuando el agente los necesita.

Hay dos tipos principales de memoria de largo plazo:

- **User profile memory**: hechos estables y preferencias, como nombre preferido o restricciones alimentarias.
- **Chat summary memory**: resúmenes destilados de temas o hilos tratados en sesiones anteriores.

Scope, identidad y aislamiento

Foundry usa el concepto de `scope` para particionar la memoria y decidir qué recuerdos pertenecen a qué usuario, sesión o recurso:

- **user IDs** como identidad estable de largo plazo,
- **session IDs** como identidad efímera de conversación,
- y, en algunos casos, **resource IDs** como identificador compartido más estable. (

Para memoria por usuario final, conviene usar `scope="{{$userId}}"` y pasar `x-memory-user-id`, o en su defecto caer al identificador de Microsoft Entra del caller.

También se recomienda implementar **per-user access controls** y minimizar datos sensibles.

Qué no deberías meter en memoria

- todo el transcript sin filtrar,
- documentos corporativos enteros,
- secretos o credenciales,
- información sensible no necesaria,
- estado técnico efímero del workflow.

Conceptos clave

> **Foundry Memory**
>
> Capacidad gestionada de **long-term memory** en Foundry Agent Service, actualmente en **preview**. Permite continuidad entre sesiones, dispositivos y workflows.

> **Memory store**
>
> Capacidad gestionada de **long-term memory** en Foundry Agent Service, actualmente en **preview**. Permite continuidad entre sesiones, dispositivos y workflows.

Scope

Clave o namespace para particionar la memoria y aislarla por usuario, sesión o recurso. Es central para seguridad y multiusuario.

User profile memory

Hechos y preferencias relativamente estables del usuario. Se recomienda recuperarla al principio de la conversación.

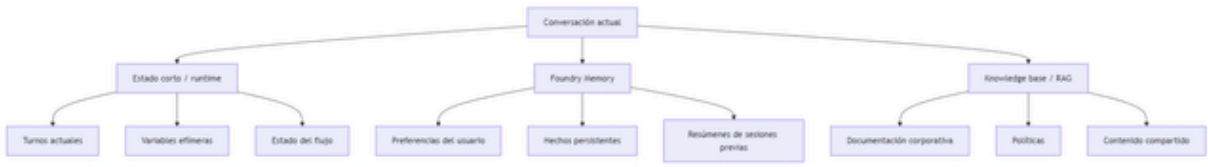
Chat summary memory

Resúmenes destilados de hilos previos, útiles para continuar conversaciones sin repetir todo el contexto.

Update delay

Tiempo de inactividad antes de actualizar memoria cuando usas la memory search tool.El valor realista por defecto es **300 segundos**.

Reparto correcto entre estado, memoria y knowledge



MermaidViewer.com

Ejemplo: crear un memory store en Foundry

az108.py

```
import os

from azure.ai.projects import AIProjectClient
from azure.ai.projects.models import (
    MemoryStoreDefaultDefinition,
    MemoryStoreDefaultOptions,
)
from azure.core.exceptions import ResourceNotFoundError
from azure.identity import DefaultAzureCredential

endpoint = os.environ["AZURE_AI_PROJECT_ENDPOINT"]
```

Ejemplo: usar Foundry Memory con LangChain/LangGraph

```
from langchain_azure_ai.chat_message_histories import
AzureAIMemoryChatMessageHistory
from langchain_core.chat_history import InMemoryChatMessageHistory
from azure.identity import DefaultAzureCredential

endpoint = "https://<resource>.services.ai.azure.com/api/projects/<project>"
credential = DefaultAzureCredential()
store_name = "customer-memory-store"

def get_session_history(user_id: str, session_id: str):
    return AzureAIMemoryChatMessageHistory(
```

Ejemplo: adjuntar memoria a un prompt agent

```
from azure.ai.projects.models import MemorySearchPreviewTool,
PromptAgentDefinition

scope = "user_123"

tool = MemorySearchPreviewTool(
    memory_store_name="customer-memory-store",
    scope=scope,
    update_delay=300,
)
```

Riesgos y límites

1. Está en preview

2. Requiere modelos compatibles

3. Hay riesgos de prompt injection y memory corruption

4. La consolidación puede cambiar durante preview

Ejercicio. Refactor de agente con historial

Añadir Store al Agente del Ejercicio Anterior

Seguridad e identidad enterprise para trabajar con datos y usuarios corporativos

En una demo, la identidad suele resolverse con una API key y un par de variables de entorno.

En producción, se recomienda: **Microsoft Entra ID** para autenticación, **RBAC** para autorización de mínimo privilegio y **managed identities** para acceso servicio a servicio sin secretos embebidos.

La base correcta: Entra ID, RBAC y managed identity

Entra ID aporta acceso condicional, managed identities, least-privilege RBAC, auditoría por principal y control de ciclo de vida del acceso, mientras que las API keys son estáticas, más difíciles de auditar y no expresan identidad de usuario.

Foundry separa dos ámbitos de permisos:

- **Foundry resource** como marco administrativo, de seguridad y monitorización.
- **Foundry project** como subámbito donde se organizan trabajo, APIs, tools y workflows. Microsoft documenta ambos scopes explícitamente en su guía de RBAC

Se empezaría con el rol **Azure AI User** tanto p[illegible] **usuario** como para la **managed identity del proyecto**, y elevar solo cuando el caso realmente lo necesite.

Usuarios corporativos vs identidades de servicio

1. Identidad de usuario humano

Sirve para autenticar al empleado o al usuario final y aplicar políticas como MFA, conditional access y permisos delegados. Se recomienda Entra ID precisamente para poder aprovechar estas capacidades en producción.

2. Identidad del servicio o del agente

Sirve para que el backend o el agente acceda a otros recursos sin almacenar secretos. Foundry soporta **managed identities** para acciones de data plane y Microsoft recomienda usarlas en lugar de service principals cuando sea posible

Microsoft está introduciendo el marco **Agent ID** en Entra ID, actualmente en **preview**, con conceptos como **agent identity** y **agent identity blueprint**. Este modelo ayuda a distinguir operaciones realizadas por agentes frente a identidades humanas o de workload, a dar acceso ajustado al agente y a gobernar agentes a escala.

También contempla un modo **attended** mediante **OAuth 2.0 On-Behalf-Of (OBO)**, en el que el agente opera con permisos delegados del usuario autenticado.

Acceso seguro a datos corporativos y tools

Para conectar un índice de **Azure AI Search** a agentes de Foundry con autenticación sin claves, Microsoft exige asignar a la **managed identity del proyecto** los roles **Search Index Data Contributor** y **Search Service Contributor**.

Para secretos y conexiones, Foundry usa por defecto un **managed Azure Key Vault** no visible en tu suscripción cuando no configuras tu propio vault. Si quieres controlar tú los secretos, puedes conectar tu **Azure Key Vault**.

Los roles mínimos válidos para ello son **Key Vault Secrets Officer**, **Key Vault Contributor** o **Key Vault Administrator**, y se recomienda normalmente el primero como permiso mínimo.

Seguridad de red y exfiltración de datos

También hay que controlar **por dónde** puede salir o entrar el tráfico. Microsoft Foundry documenta una **managed virtual network** en **preview** que permite crear private endpoints hacia servicios dependientes y restringir a qué recursos pueden acceder los agentes, con el objetivo explícito de reducir riesgo de **data exfiltration** sin obligarte a montar tu propia VNet completa.

Además, Microsoft soporta **Network Security Perimeter (NSP)** para Foundry, también en **preview**, con reglas de acceso inbound y outbound y logging centralizado de decisiones de acceso a red. Esto ayuda a contener el tráfico dentro de un perímetro lógico y a reducir exposición pública innecesaria.

Seguridad frente a prompt injection y uso indebido

Cuando un agente trabaja con documentos, webs o respuestas de tools, el riesgo ya no es solo "quién accede", sino también "qué instrucciones maliciosas entran al sistema".

Los **Prompt Shields** son parte del sistema de guardrails y controles de Foundry. Estos shields detectan y previenen intentos de manipular el comportamiento del modelo mediante:

- **user prompt attacks**, y
- **document attacks**, es decir, instrucciones ocultas en documentos, correos o páginas web que intentan secuestrar la sesión del modelo.

Conceptos clave

Microsoft Entra ID

Método recomendado por Microsoft para autenticación en producción en Foundry. Aporta OAuth 2.0, conditional access, MFA, managed identities y RBAC granular.

API keys

Mecanismo simple y útil para prototipos, pero con acceso grueso, sin identidad de usuario y con peor capacidad de auditoría. Microsoft no lo recomienda como enfoque principal para cargas enterprise.

RBAC

Modelo de autorización basado en roles y scopes. En Foundry, los scopes principales son **resource** y **project**.

Managed identity

Identidad de Azure administrada por la plataforma para acceso servicio a servicio sin secretos embebidos. Microsoft recomienda usarla cuando sea posible.

OBO (On-Behalf-Of)

Flujo delegado en el que el agente actúa con permisos del usuario autenticado, útil para escenarios attended y acceso a recursos autorizados para ese usuario. En el contexto de Agent ID está en preview.

Prompt Shields

Control de seguridad para detectar ataques de prompt injection tanto del usuario como de documentos o contenido de terceros.

> Usa **Entra ID** para producción.
> Usa **RBAC** para mínimo privilegio.
> Usa **managed identity** para acceso servicio a servicio.
> Usa **guardrails** para protegerte de lo que entra por prompts, documentos y tools.

Limitaciones actuales para decidir con criterio cuándo encaja y cuándo no

> **Foundry ya es válido para escenarios core de producción, pero no todo el stack tiene el mismo nivel de madurez ni el mismo nivel de soporte.**

El **nuevo portal de Microsoft Foundry está en GA (General Availability) para escenarios core con Foundry projects**, pero **otras capacidades quedan fuera del alcance GA o siguen evolucionando**, y que para ciertos workflows todavía puede hacer falta recurrir a la experiencia clásica o mantener rutas de continuidad.

Además, el propio servicio de agentes distingue entre capacidades **GA** y subcapacidades **preview**, como **workflow agents**, **hosted agents** o **memory**.

Región, modelo y tools no están disponibles de forma homogénea

Foundry Agent Service solo está disponible en proyectos Foundry creados en regiones que soportan Azure OpenAI Responses API, y además **no todas las tools están disponibles en todas las regiones**.

Hay límites fijos del servicio que no se pueden ampliar

Hay límites que son **fijos** y **no se pueden aumentar**. Entre ellos, Microsoft publica:

- **10.000 archivos por agent/thread**,
- **512 MB** por archivo,
- **300 GB** de tamaño total de archivos cargados,
- **100.000 mensajes por thread**,
- **1.500.000 caracteres** por mensaje de texto,
- y **128 tools registradas por agente**.

Casos de uso reales donde la plataforma aporta valor sobre una aproximación artesanal

Caso real 1: soporte L1 con documentación de proyecto + escalado a JIRA

Un agente de soporte de nivel 1 que identifique proyecto, consulte documentación en SharePoint y, si no encuentra solución, registre la incidencia en JIRA con el detalle recopilado.

Aquí la plataforma ayuda:

- **Knowledge / grounding**: Foundry puede apoyarse en **SharePoint (preview)**, **Azure AI Search** o una capa de conocimiento gestionada como **Foundry IQ**, que Microsoft define como una knowledge layer permission-aware para datos empresariales en Azure, SharePoint, OneLake y web.
- **Tools**: el tool catalog soporta built-in tools y custom tools; entre las opciones documentadas están **Azure AI Search**, **OpenAPI tools**, **Azure Functions**, **File Search** y **SharePoint (preview)**. Para una integración con JIRA, el patrón natural sería **OpenAPI tool**, **MCP**, **Azure Function** o una acción vía integración externa.
- **Autenticación**: cuando la tool o conector lo soporta, Microsoft recomienda **Entra ID** o **managed identity** en lugar de secretos incrustados; además, cada agente puede tener una **agent identity** dedicada.
- **Operación**: el ciclo oficial de desarrollo de agentes en Foundry incluye versionado, tracing, evaluación y monitorización antes y después de publicar, lo que en soporte es especialmente valioso para depurar por qué se abrió o no un ticket y con qué contexto.

Caso real 2: agente asíncrono en background basado en eventos

El agente elige la acción y el **workflow** gestiona integración, retries, pasos largos y conectores. Logic Apps aporta **1.400+ conectores** y permite exponer workflows invocables desde agentes

Caso real 3: deep agent con múltiples ficheros, razonamiento y salida no textual

Un deep agent que reciba varios ficheros, combine conocimiento interno o skills y produzca como salida un fichero o artefacto, no solo texto.

- **File Search** para grounding sobre documentos cargados,
- **Code Interpreter** para ejecutar Python en sandbox y generar análisis o charts,
- **Custom Code Interpreter (preview)**,
- y opciones custom como **OpenAPI**, **MCP** o **Azure Functions** cuando hay que producir artefactos más específicos. También documenta que File Search y Code Interpreter suelen autenticarse automáticamente a través del servicio

Caso real 4: copiloto interno gobernado sobre conocimiento corporativo

RAG sobre conocimiento corporativo, integración con sistemas internos y construcción de agentes seguros y gobernados.

- **Usaríamos Foundry IQ** como una knowledge layer gestionada y permission-aware para datos empresariales, con **Azure AI Search** y **SharePoint (preview)** como rutas de grounding.

Ejercicio. Seleccionar Proyecto Final

Qué proyecto queéis desarrollar y qué piezas tendría

Buenas prácticas para adoptar Foundry sin perder la flexibilidad de los frameworks conocidos

La mejor forma es tratarlo como una **capa de capacidades gestionadas** que se incorpora allí donde realmente reduce fricción: modelos, agentes gestionados, tools, tracing, memory, identidad, evaluación y ciclo de vida.

No plantear la adopción como sustitución total, sino como **integración incremental**.

Conservar LangGraph, LangChain o MAF allí donde el valor está en la composición y el control del flujo. LangGraph puede seguir siendo el motor de orquestación mientras Foundry Agent Service se incorpora como backend de algunos nodos mediante `AgentServiceFactory`. Y Microsoft Agent Framework, por su parte, se presenta como el sucesor de AutoGen y Semantic Kernel, combinando type safety, estado por sesión, middleware, telemetry y workflows en grafo.

Usar herramientas gestionadas cuando reducen trabajo real

Otra buena práctica importante es **usar tools built-in o catálogo privado cuando realmente te ahorran hosting, autenticación o mantenimiento**, y reservar las tools custom para lógica de negocio diferencial.

Ejemplo: mantener LangGraph y usar Agent Service en un nodo

```
import os
from azure.identity import DefaultAzureCredential
from langchain_azure_ai.agents import AgentServiceFactory

factory = AgentServiceFactory(
    project_endpoint=os.environ["AZURE_AI_PROJECT_ENDPOINT"],
    credential=DefaultAzureCredential(),
)

support_node = factory.get_agent_node(
    name="support-agent",
```

Cuestionario Sesión 1

Haz click aquí ↗ cuando lo indique el formador

Feedback Sesión 1

Haz click aquí ↗ cuando lo indique el formador

<

Previous
Preparación Alumnos

Last updated 18 hours ago